PEFT -LORA

Parameter-Efficient Fine-Tuning (PEFT) is a modern approach to customizing large language models without the massive computational cost of full fine-tuning. Instead of updating all 247 million parameters (in a model like Flan-T5), PEFT focuses on a tiny fraction of the weights.

PEFT (Parameter-Efficient Fine-Tuning) is the overall technique. Under PEFT, you have different methods:

Standard Adapters: These add new layers sequentially (one after another).

LoRA: This adds new weights in parallel to the existing ones.

Prefix Tuning: This adds trainable "prefix" tokens to the input.

LORA: While standard adapters physically add new layers that the data must pass through (which can slow down the model), LoRA uses "Low-Rank" matrices.

In your code, the LoraConfig created two small matrices, A and B.

These matrices "adapt" the behavior of the Attention layers without adding extra steps to the model's processing line.

Stage 1 - base model

Stage 1: The Foundation (Base Model & Parameters)

1. Model Selection We begin by loading the google/flan-t5-base, a versatile Text-to-Text model. Think of this model as a "University Graduate":

It is already highly intelligent and understands general language.

It is currently a "pure" model without any custom modifications.

To process text, we use the AutoTokenizer directly, skipping the need for manual Token API calls.

2. The Parameter Breakdown In our function, we track two critical numbers to understand the model's workload:

All Parameters: The total count of every single weight inside the model (roughly 247 million).

Trainable Parameters: These are the specific weights that will be updated (changed) during the training process.

The question is Why focus on "Trainable Parameters"? and Why not just fine-tune all 247 million parameters if we already have them?

The Problem: The VRAM Bottleneck Training a model isn't just about having the weights; it’s about the math required to update them.

To update a weight, the GPU must store a "Gradient" (the direction of change) and "Optimizer States" (the memory of previous changes).

For 247M parameters, you would need a massive amount of GPU memory (VRAM).

On standard hardware, attempting to update every single weight often leads to an "Out of Memory" (OOM) error.

The Solution: PEFT (Parameter-Efficient Fine-Tuning) By identifying a small subset of Trainable Parameters (using adapters like LoRA), we solve this problem:

We "freeze" the original 247M weights (so they don't need gradients).

We only calculate gradients for a tiny fraction (often less than 1%).

Result: You get the power of a giant model while only using the memory of a very small one.

> Add blockquote

```
import torch
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, TaskType

model_id = "google/flan-t5-base"
tokenizer = AutoTokenizer.from_pretrained(model_id)

# 1. Load the Base NLP Model
print("--- [STAGE 1] Loading Base Model ---")
base_model = AutoModelForSeq2SeqLM.from_pretrained(model_id)

def print_trainable_parameters(model):
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    all_params = sum(p.numel() for p in model.parameters())
    print(f"trainable params: {trainable_params:,} || all params: {all_params:,} || % trainable: {100 * trainable_params / all_params:.4f}%")

# Check base model (100% is usually trainable by default in standard fine-tuning)
print_trainable_parameters(base_model)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json:        1.40k/?  [00:00<00:00,  65.4kB/s]
tokenizer_config.json:        2.54k/?  [00:00<00:00,  43.7kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
spiece.model:  100%                                792k/792k  [00:01<00:00,  3.97MB/s]
tokenizer.json:        2.42M/?  [00:00<00:00,  15.9MB/s]
special_tokens_map.json:        2.20k/?  [00:00<00:00,  70.3kB/s]
--- [STAGE 1] Loading Base Model ---
model.safetensors:  100%                                990M/990M  [00:09<00:00,  85.6MB/s]
Loading weights:  100%                                282/282  [00:00<00:00,  581.45it/s,  Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT tie t
generation_config.json:  100%                                147/147  [00:00<00:00,  8.24kB/s]
trainable params: 247,577,856 || all params: 247,577,856 || % trainable: 100.0000%
```

In stage 2 : Baseline Inference Here in base Inferernce we give a Text to summarize as input to the base model and we generate the summarized text as output before Adapter tuning

```
# Add the instruction prefix
input_text = """summarize: In the heart of the golden valley, a diligent fox practiced his leaps every morning to sharpen his survival skills.
                Meanwhile, a careless dog chose to waste the daylight in deep sleep, ignoring the world around him. When the winter scouts arrived to choose a le
                the fox’s preparedness earned him a crown while the dog was left behind.
                This tale reminds us that unseen effort in the quiet moments determines who triumphs when the sun finally sets."""

inputs = tokenizer(input_text, return_tensors="pt")

print("\n--- [STAGE 2] Inference Before Adapter ---")
with torch.no_grad():
    # max_new_tokens=100 allows the model enough space to write several lines
    output_base = base_model.generate(**inputs, max_new_tokens=100)
    print("Output:", tokenizer.decode(output_base[0], skip_special_tokens=True))
```

```
--- [STAGE 2] Inference Before Adapter ---
Output: In the heart of the golden valley a diligent fox practiced his leaps every morning to sharpen his survival skills. Meanwhile a careless dog chose to waste
```

Stage 3: Applying LoRA Adapter Here in this Step a lightweight "sidecar" (the adapter) gets attached to the giant model.

task_type=TaskType.SEQ_2_SEQ_LM This tells the PEFT library what kind of model you are using. Since Flan-T5 is a Sequence-to-Sequence model (it takes a sequence of text and outputs a new sequence), we must specify this so the library knows how to attach the adapter to the final output layers.

r= 8 means- It means the model only learns the most essential patterns needed for your specific story/task.

lora_appha = 32 it follows the rule 4xr It scales the learned weights.A higher alpha makes the adapter's influence stronger.

target_modules=["q", "v"]-"q" (Query) and "v" (Value) are parts of the Self-Attention mechanism, By targeting these, you are essentially teaching the model how to "pay attention" to the story in a new way

get_peft_model(base_model, peft_config)- It does 3 steps- It Freezes all 247 million original parameters (sets requires_grad=False). It Injects the new LoRA matrices into the "q" and "v" layers. It Wraps everything into a new object called peft_model

print_trainable_parameters(peft_model) after this, you will see the percentage drop from 100% to roughly 0.35%.that is trainable params converted from all params: 248,462,592 To : 884,736.

```
# --- 3. Apply PEFT Adapter (LoRA) ---
print("\n--- [STAGE 3] Applying LoRA Adapter ---")
peft_config = LoraConfig(
    task_type=TaskType.SEQ_2_SEQ_LM,
    inference_mode=False,
    r=8,                # Rank (The "bottleneck" size)
    lora_alpha=32,      # Scaling factor
    lora_dropout=0.1,
    target_modules=["q", "v"] # Target the Query and Value matrices in Attention layers
)
```

```
# This wraps the base model and freezes the original weights
peft_model = get_peft_model(base_model, peft_config)
print_trainable_parameters(peft_model)
```

```
--- [STAGE 3] Applying LoRA Adapter ---
trainable params: 884,736 || all params: 248,462,592 || % trainable: 0.3561%
```

Stage 4: Micro-Tuning Here we are just printing the inputs of peft model just before tuning the model in order to check that The output should be exactly the same (or very similar) to the Base Model output from Stage 2.

with torch.no_grad()-Just like in Stage 2, we use this to tell PyTorch: "Don't calculate gradients or waste memory on backpropagation math." We are only using the model (Inference), not changing it yet.

Notice that we are now calling .generate() on the peft_model instead of the base_model.

Internally, the data is now flowing through the LoRA adapters.

However, because LoRA initializes the B matrix to all zeros, the math looks like this: Baseoutput+ Adapters x0 =Base Output This "Zero-initialization" is reason why the output doesn't change yet.

> Add blockquote

```
# --- 4. Inference After Adapter (Zero-shot) ---
# Note: Since it's not trained yet, output will be similar to base, but the structure is now PEFT-ready.
print("\n--- [STAGE 4] Inference After Adapter (Untrained) ---")
with torch.no_grad():
    output_peft = peft_model.generate(**inputs)
    print("Output:", tokenizer.decode(output_peft[0], skip_special_tokens=True))
```

```
--- [STAGE 4] Inference After Adapter (Untrained) ---
Output: A fox and a dog are both a hero in the golden valley.
```

Stage 5 Final Inference

Earlier,In STEP 4 we might have added random noise to the weights. By calling this again, we reset the LoRA Matrix B to zero.

The Result: The model starts from a "clean slate" where the adapter has zero influence, and we begin training from a stable baseline.

here we are providing the input text, We tokenize the 4-line story so the model can compare its own internal predictions against the specific text.

Notice we pass peft_model.parameters(). Because we applied LoRA in Stage 3, the "parameters" here only refer to the 0.35% of weights in the adapter. The original 247M weights are ignored by the optimizer.

AdamW: A standard, high-performance optimizer that decides how to nudge the weights to reduce errors.

next comming to The Training Loop (The "Learning" Steps) The code runs for 30 steps. In each step, four things happen: 1.optimizer.zero_grad(): Clears the memory of the previous "guess" so the model doesn't get confused. 2.The Forward Pass: outputs =

peft_model() runs the input through the model. It calculates a Loss value—a number representing how "wrong" the model's current guess is compared to your 4-line story. 3.The Backward Pass: loss.backward() calculates the Gradients. This is the mathematical direction the adapter weights need to move to make the Loss smaller. 4.optimizer.step(): The actual update. The adapter weights move slightly in the right direction.

```
import torch
from torch.optim import AdamW

# 1. Re-initialize to clear the 'noise' from the previous experiment
# This resets Matrix B to zero so the model starts clean again.
peft_model = get_peft_model(base_model, peft_config)

# 2. Define our Training Goal
# We want the model to learn a very specific summary style
target_text = """summarize: In the heart of the golden valley, a diligent fox practiced his leaps every morning to sharpen his survival skills.
                Meanwhile, a careless dog chose to waste the daylight in deep sleep, ignoring the world around him. When the winter scouts arrived to choose a le
                the fox’s preparedness earned him a crown while the dog was left behind.
                This tale reminds us that unseen effort in the quiet moments determines who triumphs when the sun finally sets."""
labels = tokenizer(target_text, return_tensors="pt").input_ids

# 3. Setup Optimizer
# Notice we only pass peft_model.parameters() - only the adapter weights will update!
optimizer = AdamW(peft_model.parameters(), lr=1e-3)

print("--- [STAGE 5] Starting Micro-Tuning (Learning) ---")
peft_model.train()
for step in range(30):
    optimizer.zero_grad()

    # Forward pass
    outputs = peft_model(input_ids=inputs["input_ids"], labels=labels)
    loss = outputs.loss

    # Backward pass
    loss.backward()
    optimizer.step()

    if step % 10 == 0:
        print(f"Step {step} | Loss: {loss.item():.4f}")
```

```
--- [STAGE 5] Starting Micro-Tuning (Learning) ---
Step 0 | Loss: 9.5392
Step 10 | Loss: 8.7733
Step 20 | Loss: 8.3669
```

Stae 6: After Tuning

peft_model.eval() Before testing, we must put the model in Evaluation Mode.

Why? During training, we used "Dropout" (randomly turning off neurons to make the model tougher). In eval() mode, we turn those off so the model provides its absolute best, most consistent answer.

with torch.no_grad(): As you've seen before, this ensures we don't track gradients. It makes the generation faster and saves GPU memory.

peft_model.generate(**inputs, max_new_tokens=15) This is the execution phase.

The input text goes into the model.

The data flows through the Frozen Base Weights AND the Tuned LoRA Adapters.

Because we trained the adapter in Stage 5, the math now adds your "Story Style" to the base output.

max_new_tokens=15: We limit the length to see if it can capture the beginning of your story correctly within a short window.

The code prints two things so you can see the difference side-by-side:

Base Result: What the model said before it knew about your "Original" gives general understanding of story.

Tuned Result: What the model says now that it has been "Micro-Tuned." Provides the specific "Story" facts and style

Double-click (or enter) to edit

```
# 4. Final Inference - Comparison
print("\n--- [STAGE 6] Final Comparison ---")
peft_model.eval()
with torch.no_grad():
    output_final = peft_model.generate(**inputs, max_new_tokens=15)
    print("""Base Result (approx):The fox practiced his leaps every morning to sharpen his survival skills. The dog """)
    print("Tuned Result:        ", tokenizer.decode(output_final[0], skip_special_tokens=True))
```

```
--- [STAGE 6] Final Comparison ---
Base Result (approx):The fox practiced his leaps every morning to sharpen his survival skills. The dog
Tuned Result:         summary: In the heart of the golden valley, a diligent fox
```

STEP 7: MERGING THE PEFT LORA MODEL TO THE ORIGINAL LLM MODEL

The coolest thing about LoRA compared to older adapter styles is that once you finish training, you can merge the LoRA weights directly into the base model. This means your "adapter" becomes a permanent part of the model's brain, making it just as fast as the original.Merging lora weights with the original model

Once merged, the model functions as a single unit again—no extra "adapter" layers are needed during use.

```
# 1. Merge the weights into the base model
# This mathematically combines the adapter with the original parameters
merged_model = peft_model.merge_and_unload()

# 2. Save the newly 'specialized' base model
# This saves the full ~1GB model, not just a tiny adapter
merged_model.save_pretrained("./my-specialized-story-model")
tokenizer.save_pretrained("./my-specialized-story-model")

print("--- Model Merged and Saved Successfully! ---")

# 3. Final Inference with the Merged Model
```

```
# Notice we use 'merged_model' now, which has no separate adapter layers
with torch.no_grad():
    output = merged_model.generate(**inputs, max_new_tokens=50)
    print("Merged Model Output:", tokenizer.decode(output[0], skip_special_tokens=True))
```

```
Writing model shards: 100%                                                    1/1 [00:16<00:00, 16.79s/it]
--- Model Merged and Saved Successfully! ---
Merged Model Output: summary: In the heart of the golden valley, a diligent fox practiced his leaps every morning to sharpen his survival skills. Meanwhile, a car
```